

Chat with Your CSV

A Streamlit app that turns any CSV into an instant dashboard, a natural-language chat interface, and a set of auto-generated visualizations. Powered by OpenAI's chat models, with real-time streaming responses and built-in token / cost tracking.

Built as the capstone project for Section 7 of the [Express Streamlit Industry Guide](#) course on Udemy.

What it does

- **Upload any CSV** or load the bundled NYC taxi sample with one click.
- **Instant metrics dashboard** at the top: rows, columns, numeric column count, missing-values percentage. Works for any data.
- **Three tabs** organize the experience:
 -  **Chat** — ask questions in plain English, get streaming answers grounded in a statistical summary of your data. Suggested starter questions get you going fast.
 -  **Data overview** — sample rows, column profile (types, null counts, uniqueness), and numeric statistics.
 -  **Visualize** — auto-generated histograms and bar charts. Pick which columns to plot.
- **Model selector** in the sidebar: switch between **gpt-4o-mini** (cheap, fast) and **gpt-4o** (more capable, more expensive) on the fly.
- **Token + cost counter** in the sidebar tracks your spend in real time as you chat.
- **Optional shared-password gate** keeps the deployed app private to people who know the password.

No RAG, no code execution, no vector databases. Just a clean, grounded chat layer over any tabular data.

Quick start (local)

1. Clone the repo

```
git clone https://github.com/migumax/streamlit-llm-csv-chat.git
cd streamlit-llm-csv-chat
```

2. Install dependencies

```
python -m venv venv
source venv/bin/activate # on Windows: venv\Scripts\activate
pip install -r requirements.txt
```

3. Add your OpenAI API key

Copy the example secrets file and fill in your real key:

```
cp .streamlit/secrets.toml.example .streamlit/secrets.toml
```

Then open `.streamlit/secrets.toml` and replace the placeholder with your real key from platform.openai.com/api-keys.

Do not commit `.streamlit/secrets.toml`. It is already listed in `.gitignore`.

4. Run the app

```
streamlit run app.py
```

The app opens at <http://localhost:8501>.

Sample data

The repo ships with a 300-row NYC TLC yellow taxi sample at `sample_data/sample.csv`. Click **Load NYC taxi sample** in the sidebar to use it instantly.

For the full multi-million-row monthly file, download from the official source:

- nyc.gov/site/tlc/about/tlc-trip-record-data.page

Any CSV works though. Try sales data, web analytics exports, a public Kaggle dataset, or your own files.

Deploy to Streamlit Community Cloud

Streamlit Community Cloud gives you a free public URL for your app in under two minutes.

1. Push to GitHub

Note: Community Cloud currently supports GitHub only. If your main repo lives on GitLab, create a mirror on GitHub just for deployment.

```
git init
git add .
git commit -m "Initial commit: Chat with Your CSV"
git remote add origin https://github.com/YOUR-USERNAME/streamlit-llm-csv-chat.git
git push -u origin main
```

2. Connect the app

1. Go to share.streamlit.io and sign in with GitHub.
2. Click **New app**.
3. Pick your repo, branch (`main`), and main file path (`app.py`).
4. Click **Deploy**.

3. Add your API key to Cloud secrets

1. From your deployed app, click the three-dot menu → **Settings** → **Secrets**.
2. Paste:

```
OPENAI_API_KEY = "sk-your-real-key-here"
```

3. Click **Save**. The app restarts and picks up the secret.

Your app is now live at a public URL like <https://your-app-name.streamlit.app>.

Community Cloud apps are publicly accessible by default. Anyone with the URL can use your app, which means anyone can call your OpenAI key. Protect yourself by setting a hard spend cap on your OpenAI account at platform.openai.com/account/limits and keeping the model at [gpt-4o-mini](#) (the default). For an extra layer of protection, enable the password gate (next section).

Authentication (optional)

This app supports a built-in shared-password gate. When enabled, anyone visiting the app sees a sign-in screen and must enter the right password before using the chat. Anyone who knows the password gets in.

This is the right tool for "share this app with my team but not the public internet." For per-user accounts (different password per person, OAuth, etc.), see the [streamlit-authenticator](#) library or [Streamlit's native OAuth](#).

Enable the gate locally

Open `.streamlit/secrets.toml` and add (or uncomment) the password line:

```
APP_PASSWORD = "change-me-to-a-strong-password"
```

Restart the app. You'll see the sign-in screen on first load. The password is stored in plain text in `secrets.toml`, which is gitignored, so it never leaves your machine.

Disable the gate

Either remove the `APP_PASSWORD` line from `secrets.toml`, or set it to an empty string. The app falls back to its open-by-default mode.

Enable the gate on Streamlit Community Cloud

After your app is deployed, open the three-dot menu → **Settings** → **Secrets** and paste both keys:

```
OPENAI_API_KEY = "sk-your-real-openai-key"
```

```
APP_PASSWORD = "your-strong-shared-password"
```

Click **Save**. The app restarts and the gate kicks in immediately. Share the URL and the password with whoever should have access.

If you forget to add `APP_PASSWORD` to Cloud secrets but have it set locally, your deployed app will be unprotected. Always check both secrets are set before sharing the URL.

Sign out

A **Sign out** button appears in the sidebar whenever the gate is enabled. Clicking it clears the authenticated session and bounces the user back to the sign-in screen. Useful when demoing the app on a shared machine.

What this gate is and isn't

It IS	It is NOT
Protection from random visitors finding your URL	Per-user account management
A way to share with a defined group of people	Compliant with enterprise auth (SSO, MFA, audit)
Easy to rotate (change the password and reshare)	Resistant to password sharing
Free and built-in	A substitute for OAuth or proper IAM

Project structure

```
streamlit-llm-csv-chat/
├── app.py                # Main Streamlit entry point (UI +
│   tabs)
├── llm_utils.py          # Data profiling + OpenAI streaming
├── requirements.txt       # Python dependencies
├── sample_data/          # Bundled with the repo
│   └── sample.csv        # NYC taxi sample (300 rows, 34 KB)
├── data/                 # Local only – gitignored, used for
│   recording demos
│   ├── yellow_tripdata_2025-08.parquet    # Original from NYC TLC
│   │   (60 MB, ~3.6M rows)
│   ├── yellow_tripdata_2025-08.csv        # Same data exported as
│   │   CSV (367 MB)
│   ├── yellow_tripdata_2025-08_100k_SAMPLE.csv # 100k-row sample for
│   │   live demos (11 MB)
│   └── sales_v2.csv                      # E-commerce sales (21
│   │   KB) – proves the app is data-agnostic
├── .streamlit/
│   └── secrets.toml.example    # Template for local secrets
├── .gitignore
└── README.md                  # This file
```

Two Python files, ~400 lines total. Everything is small enough to read in one sitting.

The `data/` folder is intentionally NOT committed: the full month CSV alone is 367 MB, which exceeds GitHub's 100 MB single-file limit. The bundled `sample_data/sample.csv` is what ships with the repo. If you want to work with the full taxi dataset, download it directly from the [NYC TLC trip record data page](#).

How it works (under the hood)

When you load a CSV (uploaded or sample):

1. `app.py` reads it with `pd.read_csv` and caches the result.
2. Top-of-page metric cards summarize the data instantly.
3. `llm_utils.build_data_context` builds a text profile of the data: shape, column names and dtypes, `df.describe()` output for numeric columns, top value counts for categorical columns, the first five rows, and a missing-values summary.
4. That profile is injected into a system prompt.
5. Each chat message you send is combined with the system prompt and the last few turns of conversation, then sent to the model you selected (`gpt-4o-mini` by default).
6. The reply is streamed token-by-token into the chat using `st.write_stream` for a ChatGPT-like experience.
7. Token usage from the response is added to a running counter shown in the sidebar.

The model answers based on the profile, not the raw rows. That keeps the context small, the cost predictable, and the behavior grounded.

Cost

Per-message cost depends on which model you pick. With the default `gpt-4o-mini`:

- Input: ~2,500 tokens = \$0.00038
- Output: ~400 tokens = \$0.00024
- **Total per message: about \$0.0006**

With `gpt-4o` (~16x more expensive):

- Input: ~2,500 tokens = \$0.0063
- Output: ~400 tokens = \$0.0040
- **Total per message: about \$0.01**

The sidebar usage counter updates after every message so you can see exactly what you have spent. A hundred messages on `gpt-4o-mini` costs around 6 cents.

Suggested starter questions

The app shows these as one-click chips when chat is empty. They are deliberately generic so they work for any CSV, not just the taxi sample:

- *Describe this dataset in plain English.*
- *Are there any columns with missing values I should worry about?*

- *What patterns or anomalies stand out in the numeric columns?*
- *Which columns look most useful for analysis, and why?*

Extending this

A few natural directions if you want to keep building:

- **Conversation export.** Add a `st.download_button` that exports the chat as markdown or JSON.
- **Code execution.** Instead of summary-grounded answers, have the model generate pandas code, sandbox-execute it, and return the real result. Adds power and security surface area.
- **RAG over long text columns.** If your CSV has a big text column (product reviews, support tickets), add an embeddings layer so the model can retrieve relevant rows before answering.
- **Function calling / tools.** Let the model call Python functions to compute aggregates, build charts, or pull row-level data on demand.
- **More chart types.** The visualize tab uses histograms and bar charts. Add scatter plots, line charts over time, or correlation heatmaps.

These are all touched on conceptually in the course's "Where to go next" lecture.

Troubleshooting

"OpenAI API key not found" Your `.streamlit/secrets.toml` file is missing or does not contain `OPENAI_API_KEY`. Copy from the example file and paste your real key.

Streaming feels stuck The first chunk from the API can take a second or two to arrive. After that, tokens stream in real time. If the spinner sits there for more than 10 seconds, check the OpenAI status page or your network.

"Rate limit" or 429 errors Your account hit OpenAI's rate limit. Wait a minute, lower your message frequency, or upgrade your plan tier.

Upload is slow on very large CSVs `pd.read_csv` loads the whole file into memory. For files over a few hundred MB, pre-sample to a smaller size before uploading, or extend the loader to use `pd.read_csv(chunksize=...)`.

Visualize tab feels empty Auto-charts only render for numeric and categorical columns. If your CSV has only weird types (datetimes, IDs, mixed), use the column multiselects to manually pick what to plot. Or extend the app to handle datetime axes.

Token counter looks off The counter sums tokens by model, then prices each model separately. If you switch models mid-session, the cost figure reflects the actual model used at each turn.

The answers feel generic or wrong The model only knows what is in the data summary. If you ask for a specific value (a row-level fact, an exact aggregate), and that value is not inferrable from `df.describe()` or the sample rows, the model will say it cannot answer. That is by design. Upgrade to the code-execution pattern (see "Extending this" above) if you need exact answers on arbitrary questions.

License

MIT. Do whatever you want with this code.

Course

This project is the capstone for Section 7 of my Udemmy Course:

[Express Streamlit Industry Guide: App Online in < 3 hours.](#)